When the Attack Is Not an Attack

Validity Failures in LLM-Generated Red-Teaming

Yihang Jiao · yihangj3@illinois.edu

https://github.com/yhj3/counterfactual-textattack

The setting, in one example

A text classifier is a function from text to a label. Feed it a film review, it answers *positive* or *negative*.

An **adversarial attack** changes the input so the *model* changes its answer while the *correct* answer stays the same. A person reading both versions would give the same label; the model does not. That gap is the vulnerability.

TextFooler finds these by swapping words for synonyms, one at a time, choosing the words the model leans on most. In principle:

> *The movie was **excellent**, I **loved** every minute.* $\rightarrow$ *positive*
> *The movie was **superb**, I **adored** every minute.* $\rightarrow$ *negative*

In practice the output is rarely that clean. Here is a real one from my data, on a news article:

> ***Original.*** *Fears for T N pension after talks. Unions representing workers at Turner Newall say they are 'disappointed' after talks with stricken parent firm Federal Mogul.*
>
> ***TextFooler.*** *Fears for T percent pension after debate. **Syndicates portrayal worker** at Turner Newall say they are 'disappointed' after **chatter** with **bereaved parenting corporations** Canada Mogul.*

"Bereaved parenting corporations" is not something a person would write. The median GPT-2-XL perplexity of a TextFooler perturbation is **522**, where ordinary English sits in the tens. An input nobody would ever produce says little about deployment risk.

Hence the idea this paper examines: hand the attack to a language model and ask it to rewrite the whole sentence so it reads naturally, while staying adversarial. This is now a common shape for automated red-teaming — one model generating the probes that test another. **My question is what such a pipeline actually measures.**

Finding 0: rewriting is a trade, not an improvement

Rewriting is applied only to attacks that *already succeed*, so its cost is directly measurable: how many survive?

44.5%, across 32 model–attack–dataset configurations. More than half the attacks are destroyed by being rewritten. What that buys is fluency: median perplexity falls from 522 to **22.9**.

The mechanism is not mysterious. The words fooling the model are often exactly the awkward ones. Repair the English and the model stops being fooled — fluency and adversarial effect are competing for the same words.

Finding 1: label validity — the probe is not an attack

I ran two rewriting prompts on identical data: a task-specific one and a generic one. By flip rate the task-specific prompt wins two to one, **119/180 against 58/180** ($p = 1.7 \times 10^{-10}$).

Then I looked at *how* it wins. AG News is news-topic classification, and the task-specific prompt for it reads "changing topic from world to sports". It instructs the model to change the label. A real result:

> ***Original*** *(business): Yukos warns it oil output is lagging. Beleaguered Russian energy giant Yukos has warned that it will not produce as much oil as expected this year…*

Rewrite: Lakers facing setbacks in NBA title defense, LeBron's injury a major concern. *The Los Angeles Lakers are facing a difficult road ahead in their quest to defend their NBA title…*

*Classifier: business → **sports**. Cosine similarity **0.016**.*

The classifier is right. That text *is* sports news. The language model did not attack anything; it wrote a different article. But a pipeline counting flips records this as a successful attack.

The usual defence, and where it breaks

The standard fix is a semantic-similarity threshold. On AG News it works — similarity 0.016 is caught by any threshold. On sentiment it does not:

Original. *Sloppy film noir thriller which doesn't make much of its tension promising set-up.* ***(3/10)***

Rewrite. *Engaging film noir thriller that successfully builds on its intriguing premise.* ***(8/10)***

*Cosine similarity **0.964** — comfortably past any threshold.*

The verdict has been reversed, 3/10 to 8/10, and the embedding hardly moves, because both sentences are about the same film noir thriller. **Sentence embeddings track topic, not verdict.** So the check catches topic changes and waves sentiment changes through.

A referee model, and the ranking reverses

An adversarial example should fool *one* model, not persuade every model. So I re-scored every probe with an independently trained classifier of the other architecture — RoBERTa refereeing BERT's probes and vice versa.

Dataset	Prompt	Referee also flips	Mean sim.
AG News	task-specific	87.2%	0.268
AG News	generic	44.4%	0.612
IMDb	task-specific	86.0%	0.749
IMDb	generic	23.1%	0.719
SST-2	task-specific	86.7%	0.640
SST-2	generic	66.7%	0.667

86–87% of the task-specific prompt's "successes" also flip an independent model. Those are not vulnerabilities. They are inputs whose label changed. Requiring the flip to be specific to the target reverses the comparison:

Prompt	Flip rate	Genuine vulnerabilities
task-specific	**119/180** (0.661)	16/180 (0.089)
generic	58/180 (0.322)	**29/180** (0.161)

The prompt that scores best on the reported metric is the worse red-teaming prompt. And this is structural rather than accidental: the instruction most effective at producing label flips is the instruction to change the label, so any prompt search that optimizes reported attack success will find it.

Two checks, and neither is sufficient. Of the 119 flips under the task-specific prompt, 39 survive the similarity threshold, 16 survive the referee, and **only 5 survive both**. If the two checks caught the same thing, the overlap would be close to 16. It is 5, because they fail in different places:

	referee: target-specific	referee: it flips too
similarity ≥ 0.75	**5**	34
similarity < 0.75	11	69

The 34 are cases the similarity check waves through and the referee catches — the 3/10-to-8/10 review above is one. The 11 are the reverse: a world-news article rewritten into a rugby story, which similarity caught at 0.268 while the referee still answered "world". Adding one check is not enough, because each one's blind spot is the other's catch.

Counting a probe as target-specific whenever the referee does not flip is also generous: the referee may have disagreed with the target on the original text to begin with. The two agree on the original 94.4% of the time, and requiring agreement leaves **12/180** genuine vulnerabilities for the task-specific prompt against **25/180** for the generic one — the reversal widens.

Finding 2: input validity — the probe is not a valid input

RTE and QNLI take *two* sentences, a premise and a hypothesis, and ask how they relate. The format is `sentence1 <SPLIT> sentence2`. Rewriting frequently destroys it:

> ***Original.*** *`Sentence1`: Dana Reeve, the widow of the actor Christopher Reeve, has died of lung cancer at age 44... `<SPLIT>` `Sentence2`: Christopher Reeve had an accident.*
>
> ***Rewrite.*** *Dana Reeve, the widow of the actor Christopher Reeve, has died of lung cancer at age 44, according to the Christopher Reeve Foundation. Christopher Reeve had an accident.* In this revised version, the sentence has been altered to include...

The two segments are merged and a commentary sentence is appended. The classifier cannot read this as a pair at all. Across the stored corpus only 100 of 198 pair-task rewrites kept the structure; for RTE under PWWS, **0 of 31** did.

The direction of the resulting error is the interesting part. **A malformed probe never registers as a flip**, so it lands in the denominator as an attack failure:

Prompt	Well-formed	Reported if unchecked	Valid probes only
generic	0.300	12.5%	**41.7%**
structured	**0.575**	21.7%	37.7%

Stating the format explicitly nearly doubles validity ($p = 2.8 \times 10^{-5}$), though the model still violates a stated format more than 40% of the time. **Finding 1 inflates the reported number; Finding 2 deflates it. The two errors push in opposite directions, so they do not cancel** — which one dominates depends on the task, and neither is visible in a success rate.

Finding 3: generator validity — the red-teamer refuses

The generating model is itself safety-trained, and sometimes declines:

> *I cannot provide a revised text that could potentially be used to mislead or deceive, as it is unethical and goes against my programming rules...*

The refusal is then scored as though it were a probe. Of 600 generations, **37 (6.2%)** begin with a refusal, and **7 of those are counted as successful attacks** — the target assigns the refusal text a different label than the original,

at mean similarity 0.216. The target is being credited with a vulnerability to a sentence about the generator's programming rules.

It is not evenly spread over the conditions being compared. The generic prompt, which says "maintain adversarial properties to fool the classifier", is refused **11.0%** of the time; the task-specific prompts, which ask for a sentiment or topic change without naming the adversarial purpose, **0.6%**. On RTE the rate is **17.5%**, plausibly because asking a model to flip an entailment reads as a request to assert something false.

That makes it a confound in my own comparison, so I checked it: excluding refusals, the task-specific prompt retains 118/179 against 52/170 ($p = 4.2 \times 10^{-11}$), essentially unchanged. Both findings survive. But **a refusal and a failed attack look identical in the output column**, and as generators are trained to decline adversarial requests more reliably, this failure grows rather than shrinks.

Finding 4: measurement validity — the number is not about the text

The last failure leaves no trace in the outputs at all. An earlier version of this study wrote the language model's output into a `llama_text` column, then computed the reported success rate from TextAttack's `result_type` field — which describes the word-substitution attack that ran *before* rewriting. The generated text was never scored.

The check is one line: assert that the column being evaluated is the column that was generated. That this happened in a study whose entire subject was generated text is the point — it is a default of the tooling, not carelessness about the topic.

Five numbers a red-teaming report should carry

1. **Semantic similarity beside every flip rate.** On AG News, the difference between reporting 65.0% and 3.3%.
2. **The fraction of successes an independent referee reproduces.** Similarity alone misses 56% of the label changes on IMDb; a referee catches them, and here it reverses which prompt is better.
3. **The fraction of generations that are well-formed inputs.** On pair tasks, the difference between 12.5% and 41.7%.
4. **The generator's refusal rate, per condition.** Here 11.0% against 0.6% for the two prompts being compared.
5. **The number of generations the target model actually scored.** Without it, a pipeline can report a metric computed from something else entirely, and nothing in the output will look wrong.

Full paper, code, stored probes, and every result table: `https://github.com/yhj3/counterfactual-textattack`